# 추진 방향 및 아키텍처 전략

OCP 네이티브 환경에 자연스럽게 녹아드는 AI 통합

## 추진 방향

기존 OpenShift 환경을 보존하면서 OpenShift Lightspeed의 기능을 사내 환경(RAG, Agent, 보안)에 맞게 통합합니다. OpenShift Console의 공식 확장 방식을 준수하여 신규 Plugin과 AI Gateway로 **OCP에 자연스럽게 녹아드는 아키텍처**를 구현합니다.

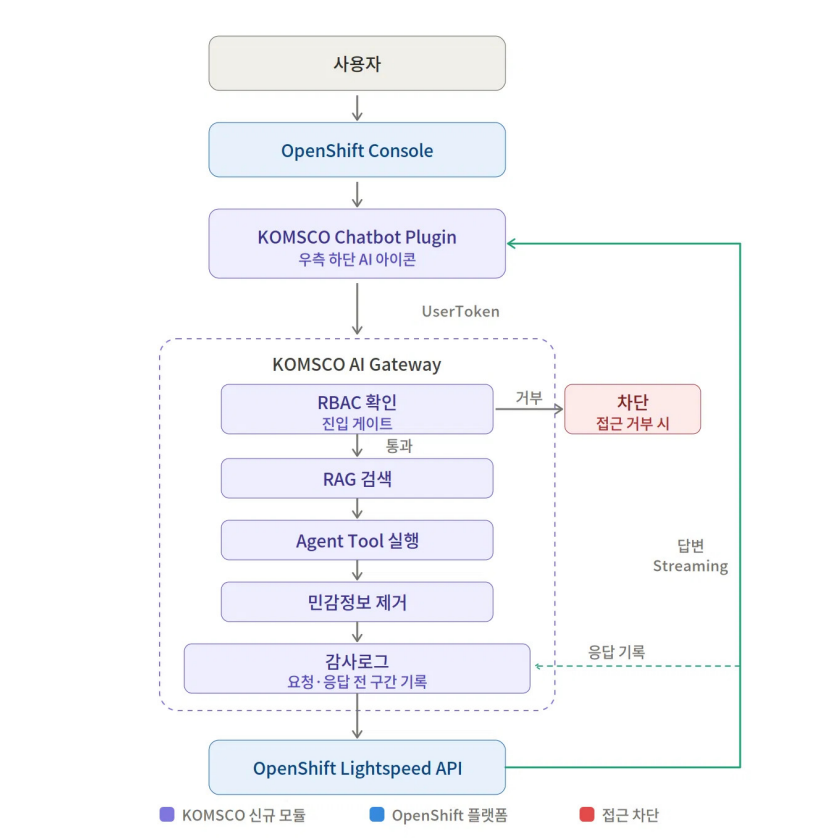
## 핵심 수행 전략

| 구분 | 수행 방안 | 기대 효과 |
|---|---|---|
| 기존 UI 처리 | Console Operator로 기존 Lightspeed plugin 비활성화 | 충돌 방지 및 단일 진입점 확보 |
| API 활용 | Lightspeed REST API(/v1/streaming_query) 연동 유지 | 성능 및 답변 품질 보장 |
| 신규 UI 개발 | Dynamic Console Plugin 기반 자체 UI 개발 | OCP 네이티브 UX 제공 및 확장성 확보 |
| Gateway 구축 | RAG, Agent Tool, 보안/감사를 전담하는 AI Gateway 배치 | 권한 통제(RBAC) 및 보안성 강화 |

**AI Gateway를 통해 RAG 연동, 감사로그 적재 및 민감정보 필터링을 수행하여 CORS, 인증, RBAC 측면에서 안정적인 통합을 제공합니다.**

# 목표 아키텍처 및 데이터 흐름

## AIOps Agentic Model 기반의 End-to-End 연동 구조




### RBAC 및 진입 통제

OpenShift UserToken 기반 인증. 권한 미달 시 AI Gateway 진입 단계에서 원천 차단.

### KOMSCO AIOps Agentic Model

특화 모델이 질문의 OS Context를 판단하고, Tool Plan JSON을 생성하여 Evidence 기반 RCA Reasoning을 수행합니다.

- ✓ OS Context 판단 (Linux/Windows/OCP)
- ✓ Tool Plan JSON 생성
- ✓ 위험 작업 승인 여부 판단

### OS-aware Tool Adapter

모델이 추상화된 Tool Plan을 내리면, Linux / Windows / OpenShift 환경에 맞는 실제 조회 명령으로 변환하여 안전하게 실행합니다.

### 안전한 Lightspeed 연동

민감정보(Secrets 등)를 제거하고 전 구간 감사로그를 적재한 뒤, Lightspeed API를 통해 최종 RCA 및 조치 가이드를 Streaming 제공합니다.

# Lightspeed + AI Gateway 연동 흐름

AIOps Model의 Tool Plan을 바탕으로 각자의 전문 영역을 조회하여 최종 답변으로 통합

# AIOps Agentic Model 적용 전략

Lightspeed가 정확한 RCA 답변을 생성할 수 있도록 Tool Plan과 Evidence Context를 구조화

## 모델 적용 위치

사용자 질문

↓

OpenShift Console Chat UI

↓

**AI Gateway**

- **KOMSCO AIOps Model**
- Evidence API / RAG 실행
- OS별 Tool Adapter 실행

## 모델 핵심 역할 정의

| 구성 요소 | 역할 | 예시 |
|---|---|---|
| **OS Context Classifier** | 대상 환경이 OpenShift, Linux, Windows 중 어디에 해당하는지 판단 | Pod 재시작, Linux 서비스 장애, Windows 이벤트 로그 장애 |
| **Tool Router** | 질문에 필요한 조회 도구를 자동 선택 | read_tool, find_tool, grep_tool, oc_tool, metric_tool |
| **Evidence Planner** | 어떤 증적을 어떤 순서로 조회할지 계획 | Event → Log → Metric → Alert → Snapshot |
| **RCA Reasoner** | 수집된 증적을 분석하여 Lightspeed에 전달할 원인 후보와 Context 정리 | OOMKilled, ImagePullBackOff, DiskPressure, Service Crash |
| **JSON Formatter** | Lightspeed가 활용할 수 있는 구조화된 Context JSON 생성 | tool_plan, evidence, root_cause, confidence |
| **Safety Guard** | 위험 명령 차단 및 승인 필요 여부 판단 | delete, patch, restart, scale 작업은 승인 필요 (차단) |

**KOMSCO AIOps Model은 Lightspeed가 최종 RCA 답변을 생성할 수 있도록 Tool Plan을 수립하고, 사내 Evidence와 OS별 Context를 구조화하여 공급하는 핵심 확장 계층입니다.**

# AIOps Agentic Model 라인업 비교

## 도입 목적에 따른 최적의 AIOps Agentic Model 선정 근거

| 비교 항목 | Qwen3.6-27B | Gemma 4 26B A4B | AIOps 적용 관점 판단 |
|---|---|---|---|
| 모델 성격 | Agentic Coding, 복합 Reasoning 강조 | 고효율 추론, Function Calling, 빠른 Triage | Qwen: 깊은 Tool Reasoning에 유리<br>Gemma: 속도·비용 효율적 |
| 구조 / 컨텍스트 | 27B 전체 파라미터 활성화 (262K) | MoE 구조 (토큰당 3.8B 활성화, 256K) | 장시간 RCA·복잡 체인: Qwen 우세<br>대량 로그 요약: Gemma 우수 |
| Tool Calling | Qwen-Agent 등 Tool Call 파서 강점 | Function Calling 및 Tool Token 체계 제공 | AIOps Tool Plan JSON은 Qwen 기반 추천 |
| OS 진단 지표 | Terminal-Bench, SWE-bench 지표 우수 | 일반 코딩/추론에 강점 | Linux 명령어 기반 추론 등은 Qwen 우선 |
| 추론 비용/속도 | 품질 우선 (긴 컨텍스트 시 GPU 부담 큼) | 속도 우선 (MoE 특성상 빠름) | 품질 중심은 Qwen, 속도/비용 중심은 Gemma 적합 |
| 학습(SFT) 용이성 | Dense 계열로 SFT 설계 상대적 단순 | MoE: Routing 안정성 고려 필요 | AIOps 전문화 출발점으로 Qwen 권장 |
| 권장 역할 | **AIOps Agentic Model**<br>(OS Tool Plan 생성, RCA JSON 생성) | **Fast Assistant Model**<br>(Alert 1차 분류, 대량 로그 요약) | 도입 환경의 우선순위(정확도 vs 효율성)에 따라 택일 |

정확한 RCA와 전문 Tool Reasoning이 핵심이라면 **Qwen3.6-27B**를, 빠른 요약·분류와 추론 효율성이 핵심이라면 **Gemma 4 26B A4B**를 선택하는 것이 적합합니다.
본 사업의 요구사항과 인프라 환경을 종합적으로 고려하여 최적의 단일 모델을 선정할 것을 제안합니다.

# OS-aware Tool Reasoning 모델 구조

동일한 장애 질문도 Linux/Windows/OCP 환경에 따라 적절한 Tool과 명령 체계를 선택

사용자 질문 → **KOMSCO AIOps Model** → **공통 Tool 선택**: `read_tool` `find_tool` `grep_tool` `event_tool` → **OS별 Tool Adapter 변환** (Linux / Windows / OpenShift)

## 대표 Tool 매핑 구조

| 추상 Tool | Linux Adapter | Windows Adapter | OpenShift Adapter |
|---|---|---|---|
| **read_tool** | cat, tail, journalctl | Get-Content, Get-WinEvent | oc get, oc describe |
| **find_tool** | find, locate | Get-ChildItem | oc get all, label selector |
| **grep_tool** | grep, rg, journalctl --grep | Select-String | log/event pattern search |
| **service_tool** | systemctl status | Get-Service | oc get co, Operator 상태 |
| **process_tool** | ps, top, pidstat | Get-Process | Pod/container 상태 |
| **metric_tool** | node exporter metric | Get-Counter | Prometheus/Thanos query |

모델은 운영 명령을 직접 실행하지 않고, **read_tool, find_tool 등 추상화된 Tool Plan을 JSON으로 생성**합니다.
실제 실행은 AI Gateway의 Tool Adapter가 담당하며, **OS별 명령 변환, RBAC 검증, 민감정보 제거, 감사로그 기록**을 수행합니다.

# AIOps Agentic Model 학습 및 고도화 전략

모델에 OS별 Tool 선택, Evidence 판단, RCA JSON 생성을 집중 학습

## 학습 가능성이 높은 이유

### 학습 범위가 명확함

일반 지식 전체가 아니라 AIOps 장애 분석, Tool 선택, JSON 출력 형식에 집중

### 운영 데이터 구조가 반복적임

로그, 이벤트, 메트릭, 알람, 장애 티켓은 패턴화가 가능

### OS별 명령 체계가 규칙적임

Linux/Windows/OpenShift의 조회 명령은 표준 패턴이 존재

### 정답 데이터 생성 가능

Runbook, RCA 보고서, 장애 조치 이력에서 Tool Plan과 RCA JSON 생성 가능

### Synthetic 데이터 확장 가능

OOMKilled, DiskPressure 등 장애 시나리오를 합성 데이터로 보강 가능

### LoRA 기반 점진 학습 가능

전체 재학습 없이 Tool Reasoning, JSON Format 중심으로 경량 튜닝(QLoRA) 가능

## 학습 데이터 구성

| 데이터 유형 | 학습 목적 및 예시 |
|---|---|
| **장애 티켓 / RCA 보고서** | 원인 분석 흐름 (증상 → 증적 → 원인) |
| **OpenShift Event/Log** | OCP 장애 패턴 (OOMKilled, Evicted) |
| **Linux 운영 로그** | OS 장애 진단 (journalctl, dmesg) |
| **Windows Event Log** | Windows 장애 진단 (App Error) |
| **Runbook / SOP** | 사내 표준 조치 및 대응 절차 학습 |
| **Tool 실행 결과 Pair** | 질문 → tool_plan → output → 판단 |
| **보안 정책 데이터** | 위험 작업 차단 (delete, patch 등) |

## 필요시 4단계 학습 방식

1. **SFT (지도학습)**
   질문, 장애 상황, OS Context, Tool Plan JSON, RCA JSON을 학습
2. **Preference Tuning**
   올바른 Tool 선택과 잘못된 Tool 선택을 비교 학습
3. **Safety Tuning**
   위험 명령, 민감정보, 권한 초과 접근을 차단하도록 학습
4. **Continuous Learning**
   감사로그, 사용자 피드백, RCA 검증 결과를 기반으로 주기적 개선

# Tool Plan JSON 기반 Lightspeed Context 강화 구조

Lightspeed가 활용할 수 있도록 Tool 계획과 증적 Context를 JSON으로 표준화

## Agent Plan JSON (Tool Plan 예시)

```
{
  "task_type": "pod_restart_rca",
  "target": { "platform": "openshift" },
  "execution_policy": { "mode": "read_only" },
  "tool_plan": [
    { "step": 1, "tool": "event_tool" },
    { "step": 2, "tool": "grep_tool" },
    { "step": 3, "tool": "metric_tool" }
  ]
}
```

## 최종 RCA Context JSON 예시

```
{
  "cause_candidates": "Deployment memory limit 감소 -> Pod OOM",
  "confidence": 0.86,
  "evidence": [ { "type": "event" } ],
  "action_candidates": [ "memory limit 복구" ]
}
```

여기서 말하는 JSON은 Lightspeed가 최종 답변을 생성하기 전에 AI Gateway가 수행할 Tool 계획, 증적 수집 범위, 위험도, 신뢰도를 구조화한 Context입니다.

# Evidence 기반 AI 장애 분석 시나리오

사용자 질문에 대해 AI Gateway가 과거 증적 기반 RCA를 수행하는 흐름

사용자 질문 (Chat UI)

**"어제 새벽에 default namespace Pod가 왜 재시작됐어?"**

## 1. Agentic Tool Plan 생성

**1 OS/OCP Context 판단**

모델이 질문을 분석하여 OpenShift Pod 재시작 RCA 질의로 분류

**2 Tool Plan JSON 생성**

event_tool, grep_tool, metric_tool, snapshot_tool 선택

**3 Tool Adapter 실행**

선택된 Tool을 환경에 맞는 명령으로 변환하여 안전하게 실행

## 2. Evidence 기반 다각도 분석

**4 과거 이벤트 조회**

해당 시간대 Pod Event, Killing, OOMKilled, Eviction 등 확인

**5 과거 로그 분석**

재시작 전후 Container Log에서 OOM 등 오류 패턴 추출

**6 장기 메트릭 분석**

CPU/Memory 사용량, Node Pressure, Restart 추세 확인

## 3. Context 구조화 및 Lightspeed 연동

**7 RCA Context JSON 생성**

수집된 증적, 원인 후보, 신뢰도, 조치 후보를 구조화

**8 Lightspeed 기반 최종 분석**

RCA Context와 사내 Runbook을 OpenShift Lightspeed에 전달하여 최종 답변 생성

**9 최종 답변 제공**

RCA, 즉시 조치, 재발 방지책, 참고 증적을 Chat UI에 제공

**본 구축안은 OpenShift Lightspeed의 기본 AI 분석 역량을 유지하면서, KOMSCO AI Gateway가 사내 Runbook/RAG, 과거 Evidence, OS별 Tool Context를 보강하여 보다 정확하고 안전한 AIOps 답변을 제공하는 확장형 아키텍처입니다.**

# 단계별 구축 로드맵 및 운영 안정성 확보 방안

## Operator/OLM 기반 Software Catalog 표준 배포 체계

### 5단계 구축 로드맵

| 단계 | 수행 작업 | 상세 내용 |
|---|---|---|
| 1단계 | 전용 Namespace 및 권한 경계 구성 | komsco-ai Namespace, ServiceAccount, 최소 권한 RBAC, NetworkPolicy, Service CA 기반 TLS 영역 확보 |
| 2단계 | AI Gateway 및 Plugin 이미지 빌드 | AI Gateway, Console Dynamic Plugin, RAG/Tool Adapter 이미지를 사내 Registry에 표준 태그로 배포 |
| 3단계 | Operator/OLM 카탈로그 패키징 | KOMSCO AI Operator를 CRD/CSV/Bundle/CatalogSource로 패키징하여 Software Catalog 설치 항목으로 제공 |
| 4단계 | CR 기반 자동 배포 및 Console 연동 | KomscoAIAssistant CR 생성 시 Operator가 Gateway, Plugin Service, ConsolePlugin CR, RBAC를 자동 구성 |
| 5단계 | UI 전환 및 운영 안정화 | 기존 Lightspeed UI 비활성화, 신규 Plugin 활성화, OLM 업그레이드 채널, 모니터링, 감사로그, 롤백 절차 안정화 |

### 운영 안정성 확보 체크리스트

| 점검 영역 | 주요 점검 및 수행 항목 |
|---|---|
| 배포/업그레이드 통제 | OLM Channel, Subscription, InstallPlan 승인 정책, CSV 상태 점검 |
| 카탈로그 관리 | 사내 CatalogSource, Bundle/Image 서명 또는 Digest 고정, 망분리 Registry 반영 |
| Operator 안정성 | CR status condition, reconcile 오류 이벤트, metrics/alert 구성 |
| Console 전환 통제 | 기존 Lightspeed Console Plugin 비활성화와 신규 Plugin 활성화를 CR 옵션으로 명시 |
| 롤백 전략 | 이전 Operator channel 또는 이전 Bundle로 복구, ConsolePlugin 원복 절차 확보 |

**Dynamic Console Plugin과 AI Gateway를 KOMSCO AI Operator로 제품화하고 Software Catalog에서 설치·업그레이드하는 방식이 OpenShift 표준 운영 모델에 가장 부합하는 안정적인 구축 방안입니다.**

# GB10 2노드 모델 구성안 3종

엔지니어 검토용 요약 · 한국어 적합성, 운영성, 토큰 생성속도를 기준으로 3개 구성안을 비교

비교 기준: 한국어 품질 | 복합 RCA 품질 | 동시성 / 처리량 | 운영 복잡도 | 향후 확장성

## 1안 혼합형

**모델 및 노드 배치**

Mistral Small 4 119B (6.5B active/token); Gemma 4 26B A4B — GB10 #1: Mistral / #2: Gemma

**예상 출력 속도**

Mistral 23–30 tok/s · Gemma 24–40 tok/s (단일 요청 기준)

### + 주요 장점

- 깊은 RCA와 빠른 일반 질의를 분리 운영
- 한국어·멀티모달·RAG·Tool Calling 균형 우수
- 업무 성격별 라우팅으로 응답속도 개선

### – 주의 / 한계

- 모델 이원화로 운영·모니터링 복잡도 증가
- 한 노드 장애 시 담당 기능·응답 품질 저하 (비대칭 Failover)

## 2안 Active-Active 이중화형

**모델 및 노드 배치**

Gemma 4 26B A4B Replica ×2 — 양쪽 동일 구성

**예상 출력 속도**

노드당 24–40 tok/s; 독립 요청 2건 합산 48–80 tok/s

### + 주요 장점

- 동일 모델 2노드 Active-Active로 단순·안정적
- 동시성·처리량·장애 대응에서 우수
- 운영 질의, 요약, 검색형 응답에 적합

### – 주의 / 한계

- 복합 RCA·긴 보고서 품질은 1안 대비 열세
- 심층 추론 작업은 별도 승격 경로 필요

## 3안 대형모델 결합형

비추천

**모델 및 노드 배치**

GB10 #1 + #2 분산 추론 결합 (TP/EP) — 제조사 공개 기준 최대 405B급 구동 범위

**검증 요구사항**

출력 성능: 모델·정밀도·분산 방식에 따라 상이, PoC 실측

### + 주요 장점

- 대형모델 구동은 기술 검증·시연 가치 존재
- 벤치마크·최대 모델 규모 검증 활용 가능

### – 주의 / 한계

- 한국어 품질 리스크·노드 간 통신 병목
- 동시성 낮고 운영 난이도 높아 주력 서비스 부적합

* tok/s는 출력 Decode 기준 예상값이며, TTFT·컨텍스트 길이·양자화 방식·추론 엔진·동시성에 따라 달라지므로 PoC에서 확정합니다.

# 구성안 상세 비교표

배치 구조, 예상 성능, 운영 특성을 표로 비교

| 항목 | 1안 혼합형 | 2안 Gemma Active-Active | 3안 대형모델 결합 |
|---|---|---|---|
| **노드 배치** | GB10 #1 Mistral / GB10 #2 Gemma | GB10 #1 Gemma / GB10 #2 Gemma | GB10 #1 + #2 분산 추론 결합(TP/EP, 모델별 적용) |
| **주요 모델** | Mistral Small 4 119B / Gemma 4 26B A4B | Gemma 4 26B A4B × 2 | 후보 예시: Llama 4 Maverick 400B, Nemotron Ultra 253B 등 |
| **단일 요청 출력 속도** | Mistral 23~30 tok/s / Gemma 24~40 tok/s | 노드당 24~40 tok/s | 참고 범위 8~20 tok/s(비보장, PoC 실측) |
| **서비스 합산 처리량** | 두 노드 동시 가동 기준, 독립 요청 2건 이론 합산 약 47~70 tok/s<br>(실제 처리량은 라우팅 비율에 따라 변동) | 독립 요청 2건 합산 48~80 tok/s | 모델 수용량 확대 목적의 분산 추론으로, 동시 처리량 증가는 제한적 |
| **한국어 적합성 사전 기대치** | 높음 | 중상 | 낮음 ~ 불확실 |
| **복합 RCA 품질** | 상 | 중 | 중 ~ 상, 모델 의존 |
| **운영 복잡도(낮을수록 유리)** | 중 | 하 | 상 |
| **장애 대응성** | 기능 저하형 Failover (한 노드 장애 시 서비스 유지, 품질 저하) | 동일 모델 Active-Active (무중단 우회 및 용량 축소 운영) | 하, 결합 구성 의존 |
| **적합한 용도** | 주력 운영형 | 고처리량·HA형 | 비추천 / 검증형 |

**1안은 모델 특성별 역할 분리로 품질과 속도 균형이 우수**

**2안은 동일 모델 2노드로 운영·HA 설계가 단순**

**3안은 한국어 품질·동시성·운영성에서 주력 서비스 부적합**

* tok/s는 출력 Decode 기준 예상값이며, 성능 수치와 정성 등급은 사전 아키텍처 가설입니다. 양자화 방식·추론 엔진·입출력 길이·동시성에 따라 달라지므로 PoC에서 확정합니다.

# 선정 판단표 및 적용 권장

점수는 PoC 이전 아키텍처 가설에 따른 상대평가이며, 최종 점수는 실측 결과로 확정합니다. (별점이 높을수록 유리)

| 평가 항목 | 1안 혼합형 | 2안 Gemma Active-Active | 3안 대형모델 결합 |
|---|---|---|---|
| 한국어 품질 | ★★★★★ | ★★★★☆ | ★★☆☆☆ |
| 단일 요청 출력 속도 | ★★★★☆ | ★★★★☆ | ★★☆☆☆ |
| 복합 RCA 품질 | ★★★★★ | ★★★☆☆ | ★★★☆☆ |
| 서비스 처리량 / 동시성 | ★★★★☆ | ★★★★★ | ★☆☆☆☆ |
| 운영 단순성 | ★★★☆☆ | ★★★★★ | ★☆☆☆☆ |
| 확장성 / HA | ★★★☆☆ | ★★★★★ | ★☆☆☆☆ |
| 실운영 적합도 | ★★★★★ | ★★★★☆ | ★☆☆☆☆ |

### 1안 적용 권장

- 1안을 기본 운영안으로 적용
- 심층 RCA, 보고서, 일반 질의를 역할 분리
- 초기 PoC는 라우팅 / 승격 정책 검증 중심으로 진행

### 2안 병행 검토

- 2안은 고처리량·HA 대안으로 병행 검토
- 처리량·HA·운영 단순성이 중요한 경우 적합
- 복합 RCA는 내부 상위 추론 모델로의 승격 경로 검토 가능

### 3안 제한 적용

- 3안은 주력 서비스용이 아니라 한계 검증용
- 한국어 품질·동시성 리스크 큼 (품질은 모델별 별도 검증)
- 기술 시연 또는 벤치마크 용도로만 제한 권고

**PoC 체크포인트:** 한국어 운영질의 정확도, RCA 증적 일치율, TTFT p50/p95, 단일 요청 출력 속도(Decode tok/s), 동시성 1/4/8 처리량, JSON Schema 유효율, Tool Call 정확도, Failover RTO,가중치·KV Cache 포함 메모리 여유율

# 소규모 RAG Vector 저장소 비교 및 권장 배치안

Dell Pro Max GB10 단일 장비 환경에 최적화된 자원 효율적 DB 선정 및 구성 방안

| 비교 항목 | PostgreSQL + pgvector | Qdrant | OpenSearch | Milvus |
|---|---|---|---|---|
| **동일 장비 배치 적합성** | **매우 높음** | 높음 | 낮음~중간 | 낮음 |
| **자원 사용량** | **낮음** | 낮음~중간 | 중간~높음 | 높음 |
| **검색 방식** | Exact(Flat), HNSW, IVFFlat | HNSW, Dense·Sparse Hybrid | BM25 + Vector Hybrid | HNSW, IVF, DiskANN, Hybrid |
| **메타데이터·권한 필터** | **SQL, JOIN, WHERE, RLS** | Payload Filter | Query Filter + Index/Doc/Field RBAC | Scalar Filter |
| **백업·복구** | PostgreSQL 표준 체계 | Snapshot | Snapshot | 분산 백업 구성 |
| **운영 난이도** | **낮음** | 중간 | 중간~높음 | 높음 |
| **보안·감사 요건 대응 용이성** (사전 아키텍처 평가) | **높음** | 중상 | 기존 운영 시 높음 | 현 규모에는 과도 |
| **본 사업 판단** | **기본안** | **성능 중심 대안** | 기존 구축 시만 검토 | **제외 권고** |

## 단일 장비 권장 배치 구조

**Dell Pro Max GB10**

- ├─ **LLM Inference Server**
- ├─ **AI Gateway (DB 접근 통제)**
- ├─ **Embedding / Reranker**
- └─ **PostgreSQL + pgvector**
  - ├─ Document Metadata / Chunk Text
  - └─ Embedding Vector / ACL Metadata

## 권장 시작 자원 및 정책

| | |
|---|---|
| **DB 구성** | **PostgreSQL + pgvector 단일 Primary** |
| **CPU / 메모리** | **Req 2 vCPU/4GiB, Limit 4 vCPU/8GiB** |
| **로컬 저장공간** | **NVMe 100GB (PGDATA·WAL·Index)** |
| **외부 NAS** | **원본 문서·Base Backup·WAL Archive** |
| **검색 방식** | **Exact 우선, p95 초과 시 HNSW** |
| **접근 정책** | **Gateway 전용 계정, 외부 직접접속 차단, 최소권한·TLS** |
| **백업·복구** | **PoC: Dump / 운영: Base Backup + WAL Archive** |

**[I/O 경로 분리]** 로컬 NVMe에는 운영 DB(데이터·WAL·Index)를 배치하고, 외부 NAS는 원본 문서 및 백업 저장소로 분리하여 NAS 장애가 온라인 검색에 영향을 주지 않도록 구성한다.

* 오픈소스 라이선스: PostgreSQL + pgvector — PostgreSQL License / Qdrant·OpenSearch·Milvus — Apache 2.0

**[결론]** Dell Pro Max GB10에 LLM과 함께 배치하는 소규모 RAG 환경에서는 **PostgreSQL + pgvector**를 기본안으로 적용한다. Qdrant는 전용 벡터 검색 성능이 필요한 경우 대안으로 검토하고, OpenSearch는 기존 운영 클러스터를 재사용할 수 있는 경우에만 검토한다.